



Lab 2 – Beppr

205.2 Beyond Relational Databases

Objective

The goal of this laboratory is to design and implement a graph database handling the data of a small social networking platform called Z (formerly Beppr). The objectives are threefold:

1. Acquire experience with [Neo4j](#), one of the most popular graph database systems.
2. Extract requirements for your database from an existing system.
3. Implement a database capable of interacting with that system.

This lab is designed to be carried out by groups of 3 to 4 students. Its estimated duration is 8 periods. The results of your work must be submitted on isc.hevs.ch/learn no later than Tuesday 5th May at 23:59 (CEST) in the form of a ZIP archive containing a zip archive of your entire project. Be sure to **add the name of each member of the group** in the README.md file. Only one student can submit for their whole group.

You may use LLM-powered tools (e.g., Claude, Copilot, etc.) to assist you with your tasks, **provided that**:

1. you fully understand and take complete responsibility for any code or text that you include in your submission; and
2. you clearly disclose the use of any assistant wherever appropriate (e.g., as comments next to generated source code or as footnotes after generated documentation).

Part 1 – Setting up a Neo4j database

You will need to install [Neo4j](#) to complete this lab. Fortunately, it that can be done easily using Docker. First, run this command to download an image:

```
docker pull neo4j:latest
```

Next, you can run the following commands to start a server. You can of course select a different password.

```
INSTANCE_NAME="neo4j-test-server"
PASSWORD="beydb-beppr"

docker run \
--name $INSTANCE_NAME \
-p7474:7474 -p7687:7687 \
--env NEO4J_AUTH=neo4j/$PASSWORD \
--env NEO4J_dbms_connector_https_advertised__address="localhost:7473" \
--env NEO4J_dbms_connector_http_advertised__address="localhost:7474" \
--env NEO4J_dbm \
neo4j:latest
```

This command will run a Neo4j server on port 7687 as well as a browser console on port 7474. You can access the later by navigating to <http://localhost:7474>. Use this browser to explore the capabilities of your database.

Part 2 – Designing and deploying a database

Download the archive attached to these instructions on your machine. It contains a Scala project that implements a simulator of social network interactions. The main entry point is the `Simulator` class, whose use is illustrated in the file `src/main/scala/Main.scala`:

```
1 import sns.Simulator
2
3 @main def Main =
4   val s = Simulator(seed = 1337)
5   for i <- 0 until 130 do println(s.randomEvent())
```

The above program creates an instance and simulates 130 events. The seed configures the random generator used by the simulator, so that a particular run produces always the same sequence of events.

The simulator only produces consistent sequence of events. That is, if the simulator generates an event representing the creation of a particular resource (e.g., a status update), then the dependencies of this resource can be assumed to exist.

Task 1 – Design

Observe the behavior of the simulator to understand the underlying database schema. The goal here is to collect the requirements that your database will have to satisfy. You can ask yourself the following questions to kickstart this discovery process:

- What kind of events can be generated?
- What kind of relationship can happen between the events?
- What kind of data is persisted on the system?

You can also have a peek at the code of the simulator to confirm your conclusions.

Document your findings in the `README.md` file of the project. Your documentation should be specific enough to allow another team to skip Task 1 and move straight to the implementation phase.

Task 2 – Implementation

Based on the requirements that you have collected with Task 1, design a database schema to handle the data that are managed by the social media platform and then:

1. Document your schema in the `README.md` file.
2. Implement a Neo4j database using your schema.

Your documentation should be specific enough to allow another team to understand how your database is supposed to be used.

Task 3 – Deployment

It is time to put your database to use!

In addition to the `randomElement` method, which generates a new event, the simulator also has a `challenge` method, which can be used to test your database. This method accepts a query handler in the form of a closure that accepts a query and returns a response to that query. The following illustrates how `challenge` is meant to be applied:

```
1 import sns.Simulator
2
3 def handler(query: Simulator.Query): Option[Int] =
4   if query == "count:users" then Some(0) else None
5
6 @main def Main =
7   val s = Simulator(seed = 1337)
8   s.challenge(handler)
```

Your query handler should return `None` to indicate that it did not understand the query. Otherwise, it should return `Some(n)` where `n` is the response to the query (e.g., the number of users on the system). `challenge` returns `true` iff your answer is correct.

Your work will be evaluated with the following setup:

```
1 val s = Simulator(seed = s)
2 for i in 0 until n do
3   if (i & 0b11) != 0b11 then
4     s.randomEvent()
5   else
6     s.challenge(handler)
7 print(s.score())
```

The `score` method returns a ratio between the number of queries that have been issued and the number of queries that have been answered correctly. The values of `s` and `n` can be arbitrary.